```
# ============================================================
# Question 1: Constraint Satisfaction Problem (CSP)
# Assigning Courses to Classrooms using Backtracking Search
# ============================================================

# Variables: AI, DS, CN
# Domains: Each course can be assigned to Room A, Room B, or Room C
variables = ["AI", "DS", "CN"]
domains = {
    "AI": ["Room A", "Room B", "Room C"],
    "DS": ["Room A", "Room B", "Room C"],
    "CN": ["Room A", "Room B", "Room C"]
}

# Constraints:
# 1. AI cannot be assigned to Room A
# 2. DS cannot be assigned to Room C
# 3. No two courses can be assigned to the same classroom
# 4. CN must be assigned to either Room A or Room C

def is_valid(assignment, var, value):
    """Check if assigning 'value' to 'var' violates any constraint."""

    # Constraint 1: AI cannot be Room A
    if var == "AI" and value == "Room A":
        return False

    # Constraint 2: DS cannot be Room C
    if var == "DS" and value == "Room C":
        return False

    # Constraint 4: CN must be Room A or Room C
    if var == "CN" and value not in ["Room A", "Room C"]:
        return False

    # Constraint 3: No two courses share the same room
    for assigned_var, assigned_val in assignment.items():
        if assigned_val == value:
            return False

    return True


def backtracking_search(assignment, variables, domains):
    """Recursive backtracking algorithm to solve the CSP."""

    # Base case: all variables assigned
    if len(assignment) == len(variables):
        return assignment

    # Pick the next unassigned variable
    unassigned = [v for v in variables if v not in assignment]
    var = unassigned[0]

    # Try each value in the domain
    for value in domains[var]:
        if is_valid(assignment, var, value):
            assignment[var] = value          # Assign
            result = backtracking_search(assignment, variables, domains)
```

```
            if result is not None:
                return result
            del assignment[var]             # Backtrack

    return None  # No valid assignment found


# ---------------- Run the CSP Solver ----------------
solution = backtracking_search({}, variables, domains)

print("=== CSP: Course-Classroom Assignment ===")
if solution:
    for course, room in solution.items():
        print(f"{course} -> {room}")
else:
    print("No valid assignment found.")
```

```
=== CSP: Course-Classroom Assignment ===
AI -> Room B
DS -> Room A
CN -> Room C
```

```
# ============================================================
# Question 2: Knowledge-Based System (KBS)
# Recommending Academic Status (Grade) based on Marks
# ============================================================

# Knowledge Base represented as a list of rules.
# Each rule is a (condition_function, conclusion) pair.
knowledge_base = [
    (lambda marks: marks >= 85, "A"),
    (lambda marks: 70 <= marks < 85, "B"),
    (lambda marks: 50 <= marks < 70, "C"),
    (lambda marks: marks < 50, "Fail")
]


def infer_grade(marks):
    """Apply the knowledge base rules to determine the grade."""
    for condition, conclusion in knowledge_base:
        if condition(marks):
            return conclusion
    return "Invalid marks"  # Safety fallback


def run_kbs(marks):
    """Validate input and display the result."""
    if marks < 0 or marks > 100:
        print(f"Marks: {marks} -> Invalid input (must be between 0 and 100)")
        return
    grade = infer_grade(marks)
    print(f"Marks: {marks} -> Grade: {grade}")


# ---------------- Test the KBS with sample inputs ----------------
print("=== Knowledge-Based System: Student Grading ===")

test_marks = [90, 75, 60, 40, 85, 69.9]  # at least 3 test cases as required
for m in test_marks:
    run_kbs(m)
```

```
# ---------------- Optional: take live input from user ----------------
try:
    user_marks = float(input("\nEnter student marks (0-100): "))
    run_kbs(user_marks)
except ValueError:
    print("Please enter a valid number.")
```

```
=== Knowledge-Based System: Student Grading ===
Marks: 90 -> Grade: A
Marks: 75 -> Grade: B
Marks: 60 -> Grade: C
Marks: 40 -> Grade: Fail
Marks: 85 -> Grade: A
Marks: 69.9 -> Grade: C

Enter student marks (0-100): 34
Marks: 34.0 -> Grade: Fail
```